QING TRAVELER

Hostel Discovery & Travel Community Platform

TECHNICAL PROJECT PORTFOLIO | 2026



Figure 1. Homepage destination carousel — screenshot supplied from the deployed prototype.

Project context: Digital Media Technology course group project

My contribution: Primary responsibility for product conceptualisation, requirements planning, and core front-end implementation

Development approach: Comprehensive initial prompt in Cursor, followed by iterative refinement

Technology: HTML5 · CSS3 · Vanilla JavaScript · Bootstrap · Leaflet · Swiper

Live demo: xieyanqi.github.io/hostel-discovery-platform

Public repository: github.com/xieyanqi/hostel-discovery-platform

01 Project Overview & Initial Requirements

Qing Traveler is a single-page front-end prototype designed to bring hostel discovery, travel guides, map-based browsing, traveller communities, and travel stories into one browsing experience.

The idea originated from an independent trip to Xiamen. During peak travel season, finding a hostel with a reasonable price and a good experience required searching across Mafengwo, Qunar, and Xiaohongshu. Accommodation information and traveller recommendations were fragmented. I wanted to explore whether these resources could be organised in one place.

I developed the idea as a Digital Media Technology course group project and took primary responsibility for the product concept and core front-end implementation.

Initial product brief

Before development, I prepared a comprehensive website specification covering the product concept, visual direction, information architecture, functional modules, and expected interactions. I submitted this as the initial instruction to Cursor.

Planned area	Original product intention	Current prototype evidence
Hostel discovery	Cards, region/price/type filters, detail views	Nine sample records; client-side filters; detail modal
Travel guides	Guide cards, itinerary, transport, accommodation, food	Seven sample guides; detail modal and map view
Traveller community	Regional/interest groups and activity maps	Group cards and sample event-map view
Travel stories	Image-led stories and categories	Cards populated from a fixed mock dataset
Membership concepts	Tiers, points, promotions, personalised services	Presentation UI only; no persistent membership service

The initial brief contains ideas beyond the implemented prototype. The current source code and this portfolio distinguish product proposals from delivered functionality.

02 Hostel Discovery

The hostel listing is generated from nine demonstration records stored in `main.js`. Users can filter cards by geographical region, price range, and hostel category. The project owner has tested the filtering interaction and confirmed that the displayed listing changes according to the selected conditions.



Figure 2. Hostel discovery page with region, price, and category filters.

The detail modal presents an overall rating, rating dimensions, review tags, and sample reviews. These are hard-coded examples rather than verified user-generated records.

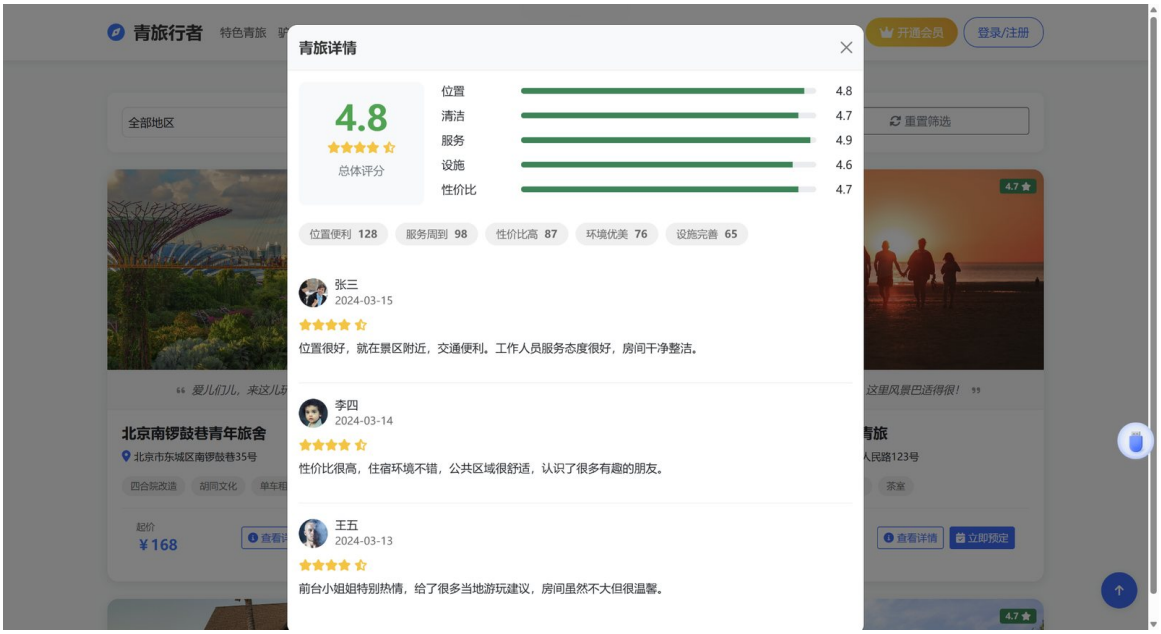


Figure 3. Hostel detail modal with sample rating and review information.

Implementation notes

`applyFilters()` reads the selected controls, filters the `allHostels` array, and calls `renderHostels()` to update the displayed cards. `showHostelDetail()` updates a Bootstrap modal using demonstration review data.

03 Travel Guides & Itinerary Details

Seven sample travel guide records are defined in `main.js`. Each guide card presents a title, author, date, summary, and engagement counts. Selecting a guide opens a detail view containing an overview, itinerary schedule, transport notes, accommodation examples, and food recommendations.



Figure 4. Travel guide listing.



Figure 5. Guide detail modal showing itinerary and supporting travel information.

The guide content is predefined demonstration data. The displayed likes and comments are sample counts, not live community engagement metrics.

04 Map-based Browsing & Traveller Community

Leaflet is used to create map views with OpenStreetMap tiles. Markers are placed using predefined latitude/longitude values. The maps visualise sample hostel, itinerary, and community-event locations; they do not provide route optimisation, turn-by-turn navigation, or live location tracking.



Figure 6. Itinerary map with predefined location markers.

The community interface presents three regional-group examples, two interest-group examples, and two organisation examples. Activity-map views display sample event locations and dates.



Figure 7. Traveller community interface.

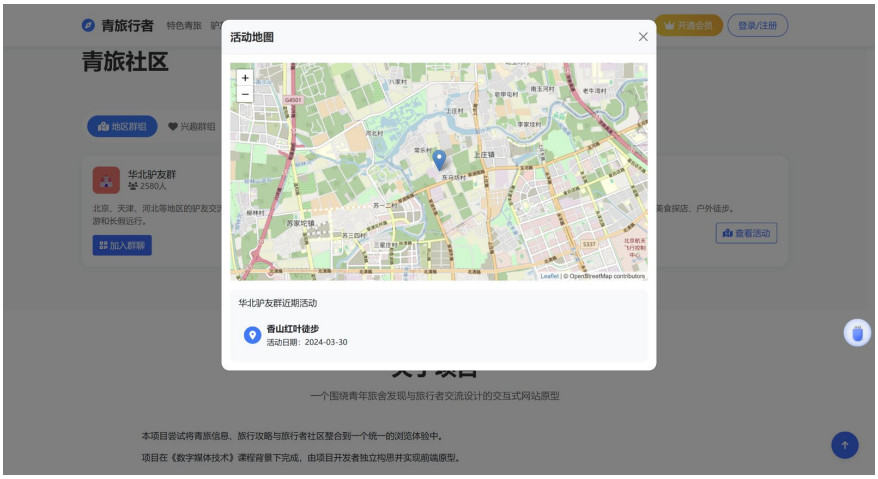


Figure 8. Community activity map.

The group-joining control is a placeholder: `showGroupQR()` only logs a placeholder QR-code path to the browser console.

05 Travel Stories & Supplementary Interfaces

The travel stories section presents image-led cards with a category, title, excerpt, author, and date. The current content is returned by a simulated asynchronous JavaScript function using a fixed sample dataset; no remote story API or persistent publishing service is connected.

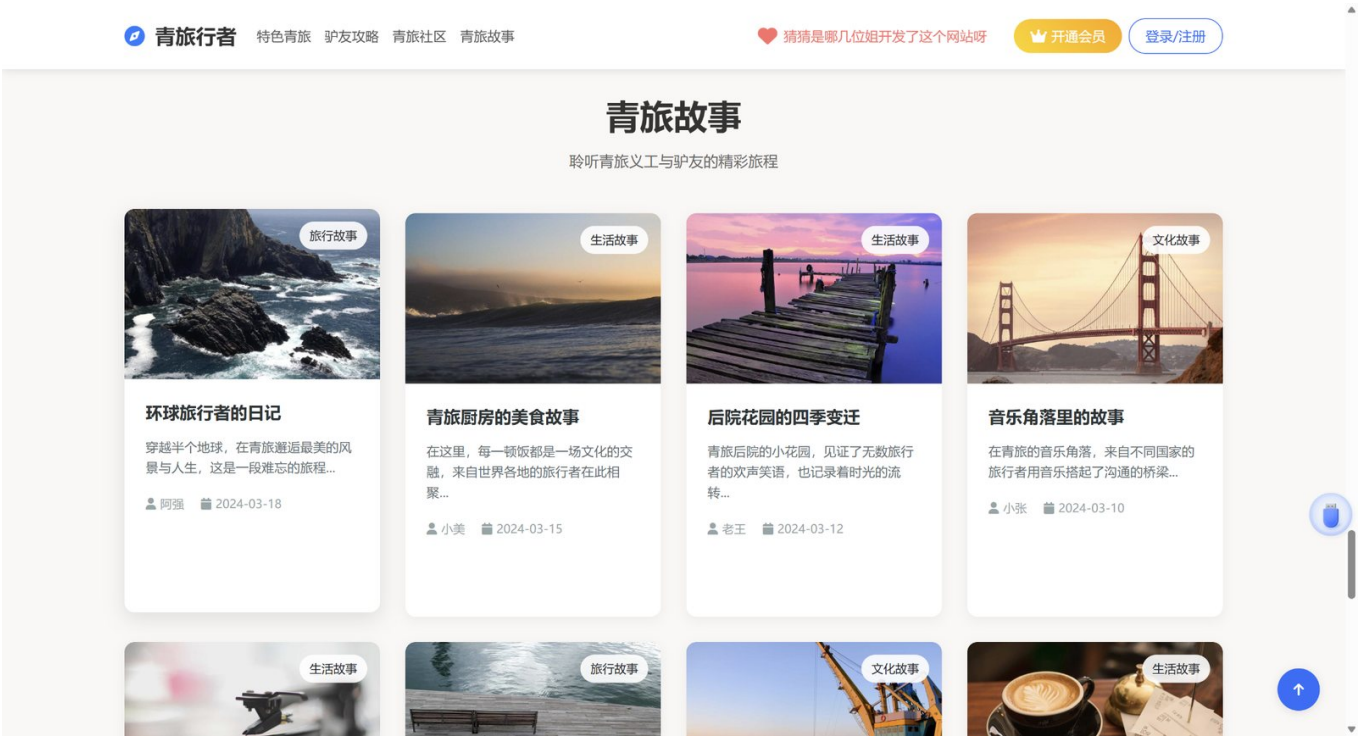


Figure 9. Travel stories card grid.

The source also contains supplementary membership and danmaku-style interaction interfaces. These are front-end demonstrations. Membership payment is simulated, and the guestbook comments initialisation references elements that are absent from the current HTML.

The screenshots in this portfolio are genuine screenshots supplied from the running website. They document the visible interface, not the existence of a production backend.

06 Technical Implementation

The project follows a lightweight static front-end structure. The page does not require a build pipeline or application framework; external libraries are loaded through CDNs.

Technology	Role in the prototype
HTML5	Single-page structure, sections, forms, and modal markup
CSS3	Layout, visual styling, responsive breakpoints, and animations
Vanilla JavaScript	Data arrays, DOM rendering, filtering, event handling, and modal updates
Bootstrap 5.1.3	Grid/layout utilities, modal components, and UI styling
Leaflet 1.7.1 + OpenStreetMap	Map containers, tiles, and location markers
Swiper 8	Homepage destination carousel
Font Awesome 6	Interface icons
Cursor	AI-assisted initial code generation and iterative code revision

Client-side data and rendering flow

Step	Implementation
1	`index.html` loads the page structure, CDN libraries, `styles.css`, and `main.js`.
2	`main.js` defines `allHostels`, `travelGuides`, `communityData`, and `carouselData`.
3	Rendering functions populate hostel, guide, and community cards from JavaScript arrays.
4	`applyFilters()` filters `allHostels` and calls `renderHostels()`.
5	Leaflet creates maps and markers from predefined coordinates; Bootstrap modals present detail views.

Repository note: `data/hostels.js` is retained but is not imported by the current page. The active hostel dataset is in `main.js`.

07 AI-assisted Development & My Contribution

I used Cursor as an AI-assisted coding environment. I first prepared a comprehensive natural-language website specification and submitted it as one initial prompt. Rather than requesting individual pages separately, the brief covered the product concept, visual direction, information architecture, major modules, and expected interactions.

Cursor generated the main website structure and code from this initial instruction. I then reviewed the generated website and continued to request modifications during development.

Development workflow

Stage	My activity
1. Conceptualisation	Identify the fragmented travel-information problem and define the platform objective.
2. Requirements	Prepare a comprehensive initial brief covering layout, modules, and interactions.
3. Initial generation	Submit the brief to Cursor to generate the main website structure and code.
4. Evaluation	Preview the result and compare the visible interface with the intended design.
5. Refinement	Provide further instructions to Cursor to modify code and improve the implementation.
6. Publication	Test selected interactions, publish the source repository, and deploy through GitHub Pages.

My role

Although the project was completed in a course group-project setting, I took primary responsibility for product conceptualisation, requirements planning, and core front-end implementation. My work included defining the product direction, preparing the initial specification, directing Cursor's code generation, evaluating the output, and guiding subsequent modifications.

The main code was generated and modified with AI assistance. My contribution should not be described as independently hand-writing every line of code.

Evidence note: the original Cursor chat history and intermediate versions were not retained. This workflow description reflects my recollection and the preserved initial product specification, not reconstructed chat logs.

08 Evaluation, Limitations & Reflection

Area	Current status
Hostel filters	Client-side region, price, and type filtering; owner-confirmed tested
Hostel and guide detail views	Implemented with demonstration content
Itinerary and activity maps	Leaflet maps and predefined markers
Group joining	Placeholder only; no working join flow
Authentication and accounts	No production authentication or persistent accounts
Booking and payment	Demonstration interfaces only; no live service
Data persistence	No server-side database

Source-review findings

The source review identified a parsing issue in the `500+` price filter, missing HTML elements referenced by the guestbook comments code, undefined secondary inline handlers, and a fixed mock dataset behind the story load-more behaviour. The payment result is simulated with a random outcome. A JavaScript syntax check (`node --check main.js`) passes, but full browser-based regression testing has not been completed.

Reflection

This project showed me how an initial product concept can be translated into an interactive prototype with AI-assisted development. It also exposed the difference between describing desired functionality and understanding how that functionality is implemented. I could define the information users should see and evaluate whether the generated interface matched my expectations, but I recognised gaps in my understanding of program logic, data organisation, and software architecture.

This experience strengthened my motivation to pursue formal Computing education. I want to move beyond directing AI tools to produce interfaces and develop the ability to understand, design, implement, test, and maintain software systems more independently.

Future learning directions

Modular data architecture · reliable event handling · backend and database fundamentals · authentication · automated testing · systematic debugging.

Live demo: https://xieyanqi.github.io/hostel-discovery-platform/

Public repository: https://github.com/xieyanqi/hostel-discovery-platform

CV-ready summary: Took primary responsibility for product conceptualisation and core front-end development of an AI-assisted travel platform prototype. Prepared a comprehensive product specification, used Cursor to generate the main website structure and code, and iteratively refined the implementation through browser-based evaluation. Implemented client-side hostel filtering and Leaflet-based map views; published the prototype through GitHub Pages.